

Memory Overload: Data Layout & Parallel Processing in Large-Scale Traffic Analysis

System 1

NYC DOT Real-Time Traffic Speed Feed · 46.2M records · C++17 + OpenMP · 3-Phase AoS → SoA Benchmark · CMPE 275 Mini 1A

46.3M

Records
Processed

79.3 s

Serial Ingest
(Phase 1)

18.2 s

Parallel Ingest
(8T)

4.4×

Ingest
Speedup

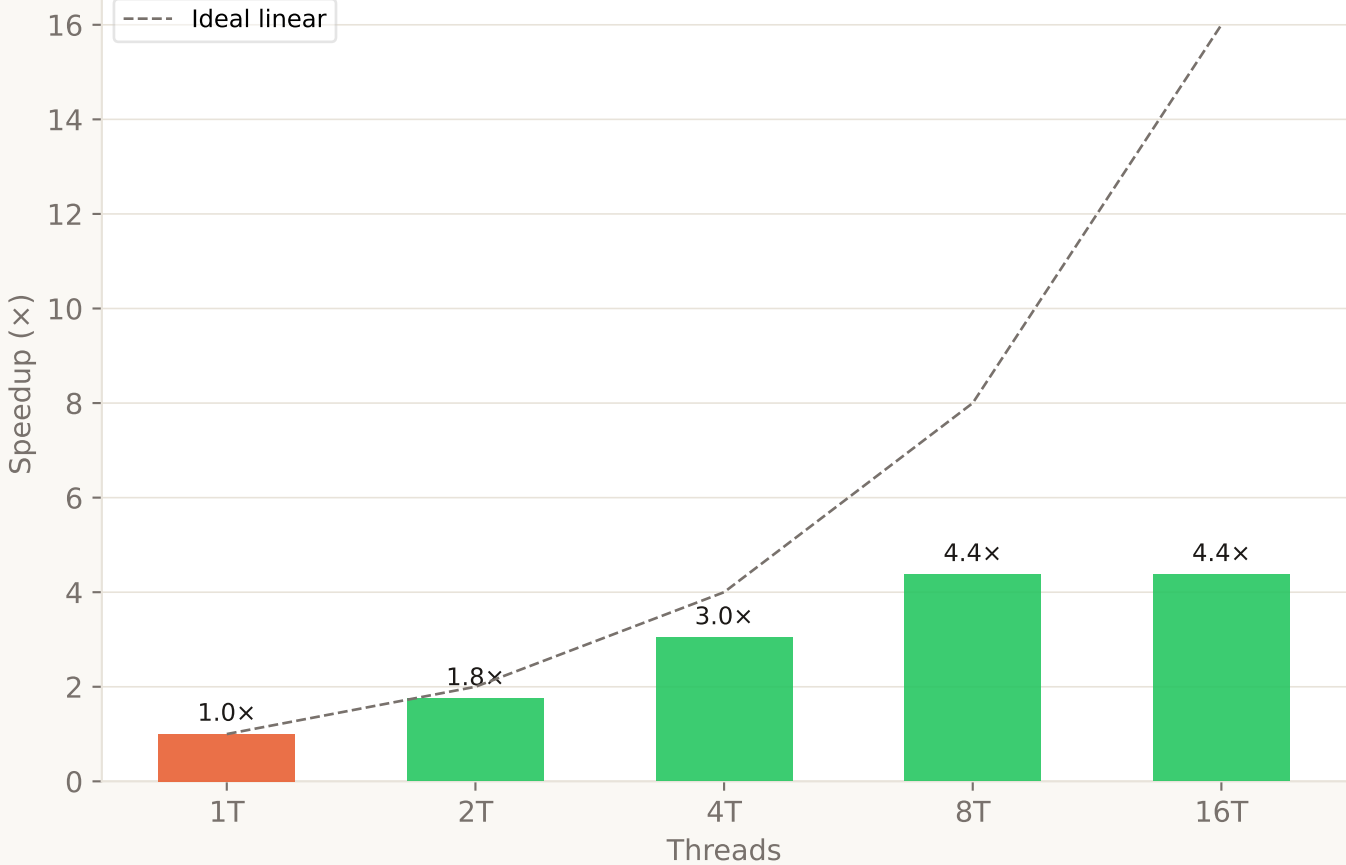
13.5×

Best Query
Speedup (W1 Speed
Threshold)

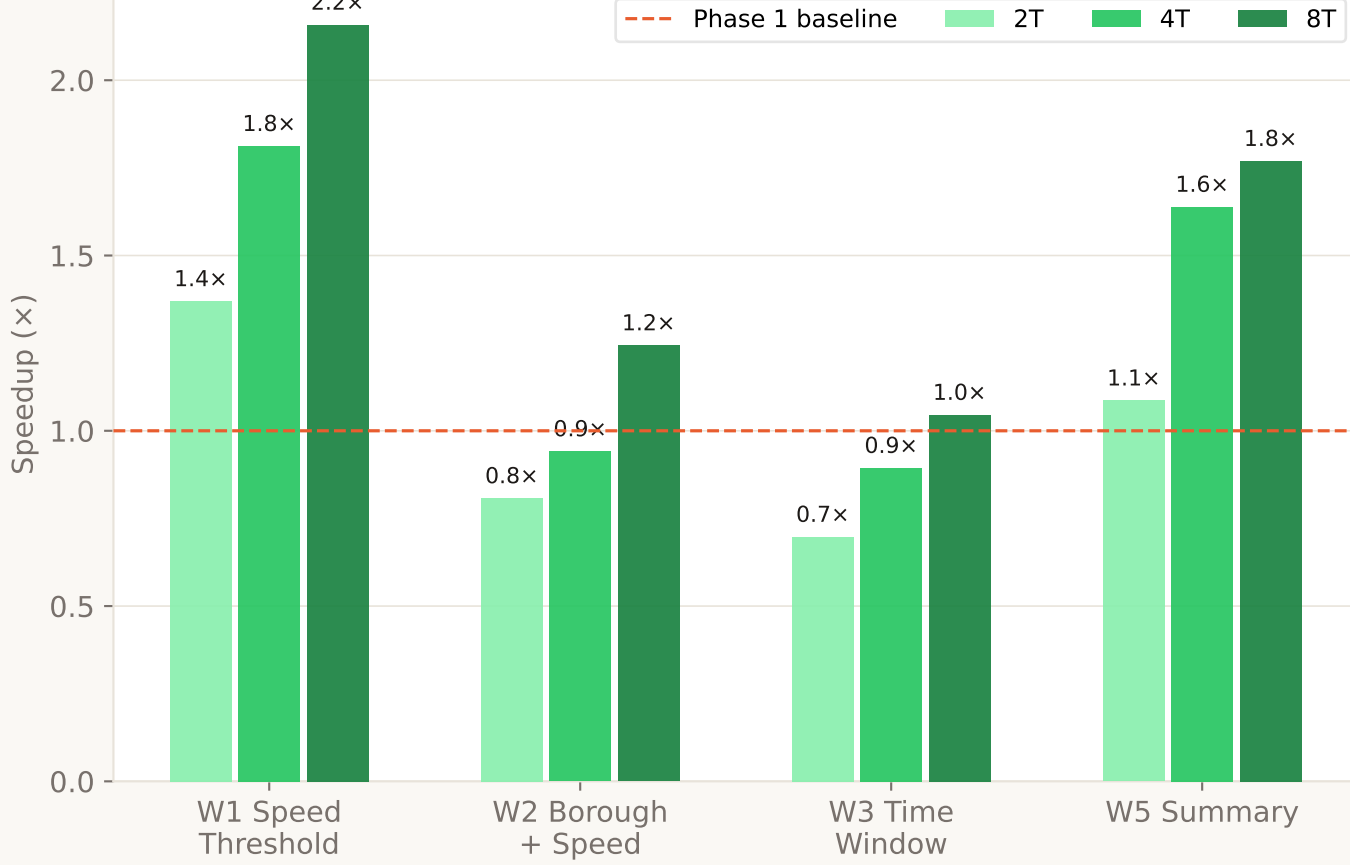
n/a

Memory
Reduction (SoA)

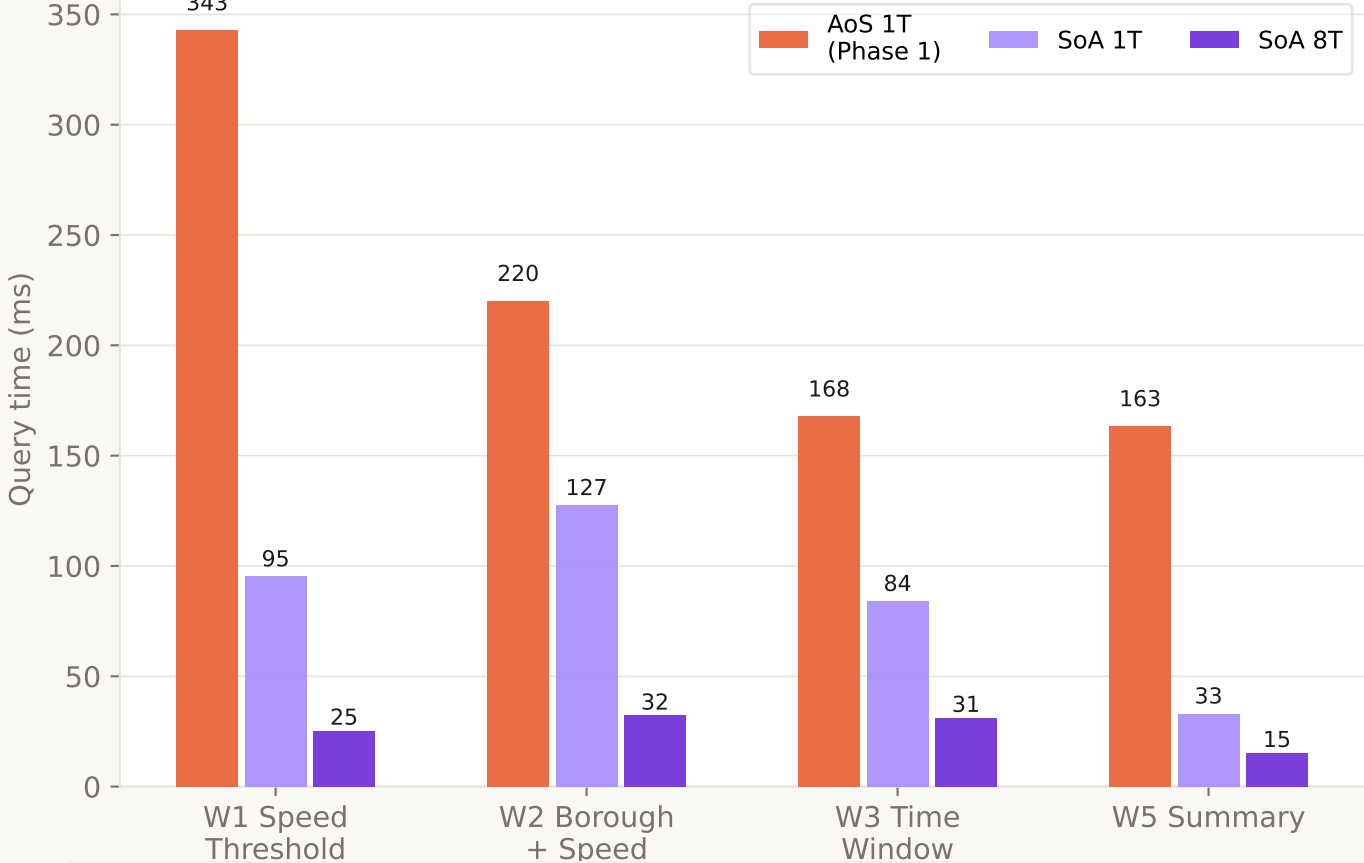
Ingest Speedup vs Serial (46M rows)



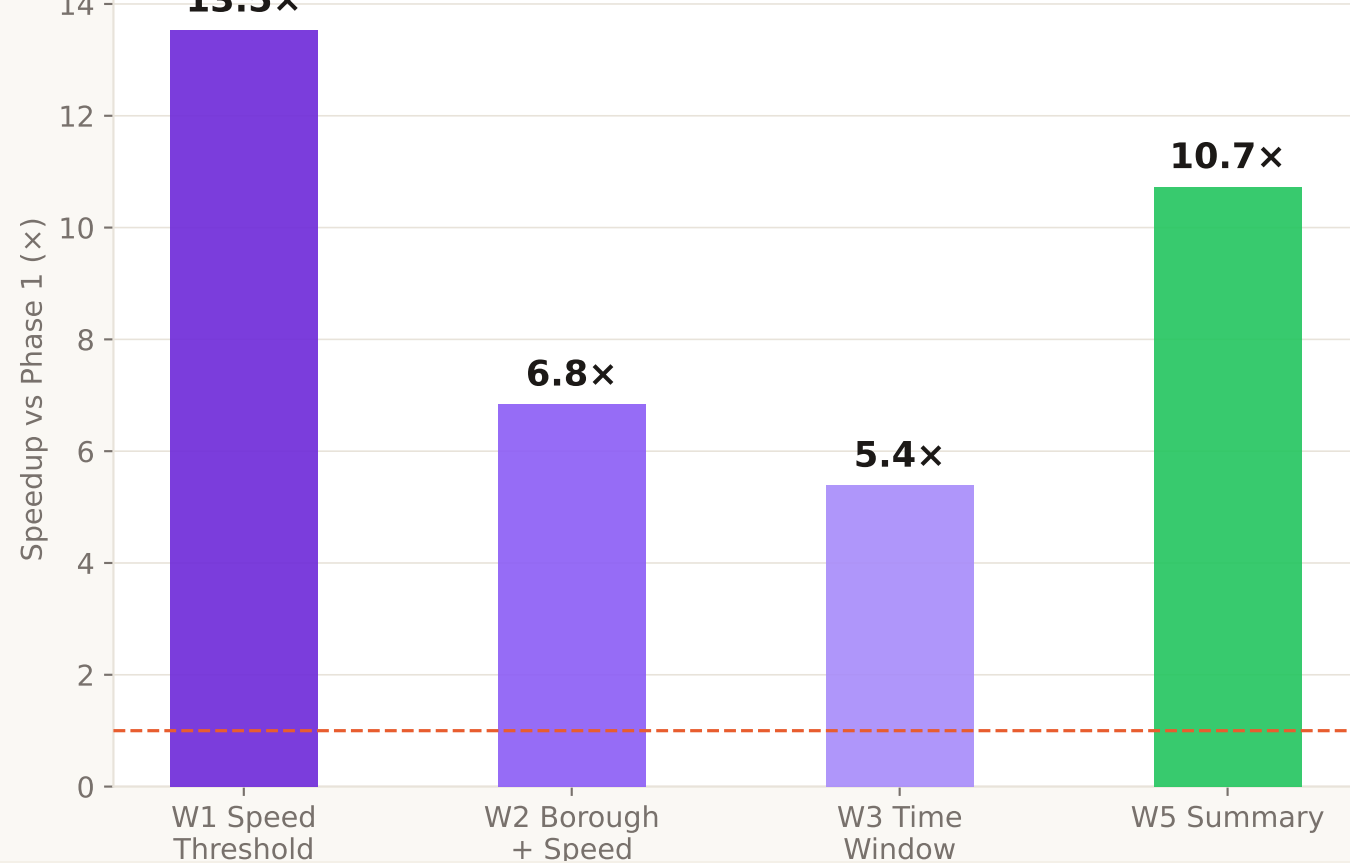
Phase 2 — Query Speedup vs Phase 1 Serial (AoS)



Phase 3 — Query Latency: AoS vs SoA (46M rows)



Total Speedup: Phase 3 (SoA + 8T) vs Phase 1 Serial



KEY FINDINGS

① Ingest Parallelism Critical at Scale

Serial CSV parsing dominates total runtime at full scale: 79.3 s ingest time versus sub-second queries. Parallel ingest at 8 threads reduces this to 18.2 s, which is a 4.4× speedup.

② SoA Layout Dominates Query Performance

AoS scans full records, while SoA streams only required columns. From the current raw batch, layout-only speedup (Phase 3 serial vs Phase 1 serial) is 1.5× – 4.9×.

③ Combined: Up to 13.5× Total Speedup

SoA + 8-thread parallelism achieves 1.3× – 13.5× over the serial AoS baseline. Best query in this run is W1 Speed Threshold at 13.5×.

④ Phase Comparison Summary

Phase 1	Serial AoS	1.0×	baseline	
Phase 2	Parallel AoS	1.0×	– 2.2×	query
	Parallel ingest	4.4×		
Phase 3	SoA serial	1.5×	– 4.9×	layout
	SoA + 8T	1.3×	– 13.5×	total

Dataset

NYC DOT Real-Time Traffic Speed Feed 46,264,669 records · 19 GB raw CSV 16-core Intel i9-12900 · 94 GB RAM GCC -O2 -fopenmp · Linux 6.8